

Day 12 — Content Modifier, Part 3 (Data Formats, XPath, Simulation & Number Range)

1. What Are the Data Formats in CPI?

Take a simple employee record as the running example:

Field	Value
Emp ID	1006
Emp Name	Bharath
Designation	Developer
Company	Infosys

The point: if we want to send the above data through CPI, how do we send it? The key rule from class: **we should not send data to the CPI system in whatever format we like — we need to send the data in the required format.**

Correction (third time this has come up — Days 7, 9 and now 12): Your notes say "**SAP CPI → required format → XML (only).**" As flagged earlier, CPI does **not** accept "only XML" as a platform. Notice that **your own notes on the next page contradict this** — they correctly say "**SAP CPI → CSV, XML, JSON (data).**" That second statement is the accurate one.

The accurate version to remember: CPI can *receive and send* CSV, XML, JSON, text and more. But many palette steps (especially graphical Message Mapping) **work on XML internally**, so the standard habit is: **convert non-XML data to XML first (Converter step), process it, then convert back if the receiver needs CSV/JSON.** That habit is what your trainer is teaching — the "XML only" wording is just shorthand for it.

2. What Is "Data Format"?

Data format = how you are transforming data from one system to another (Source → Target) and how you are managing that data. For that, we call it the **data format**.

SAP CPI handles three main data formats: CSV, XML, JSON.

3. XML — Structure and Terminology

XML is built from **root tags, parent tags, child tags** — basically **open and closed tags**.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<employee>
  <id>1006</id>
  <name>Bharath</name>
  <designation>developer</designation>
  <company>Infosys</company>
</employee>
```

Part	What It Is
<?xml version="1.0" encoding="UTF-8"?>	XML Declaration — the file's introduction, telling the reader the XML version and character encoding
<employee>	Root node (also the Parent tag)
<id>1006</id>	An element — the tag name is the element, 1006 is the value
<name> , <designation> , <company>	Sub-elements (or data elements) under the root

Reminder: every opening tag needs a matching **closing tag with a forward slash** (`</name>`). In handwriting this slash is easy to miss, but XML is strict — a missing or mismatched closing tag makes the whole document invalid, and CPI steps that parse it will fail.

4. Reading a Value from XML with XPath

Question: To read the **name** from the above XML, how do you read it?

Style	XPath
Absolute	<code>/employee/name</code>
Relative	<code>//name</code>

(Same Absolute vs. Relative idea from Day 11: single `/` = exact path from root; double `//` = find it anywhere.)

5. XML with Multiple Records — Reading the 2nd or 3rd One

Question: If the XML has multiple structures, how will you read them? **The root node comes once; under it, multiple records exist.**

```
<employees>
  <employee><name>Ashok</name></employee>
  <employee><name>Somi</name></employee>
  <employee><name>Pavan</name></employee>
</employees>
```

How to read the 2nd record's name?

`/employees/employee[2]/name` ← **very important**

Correction — important: Your notes compare this to the **Java array concept [0, 1, 2, 3]**. That comparison is misleading and will cause off-by-one mistakes. **XPath indexing starts at 1, not 0** (this is the official XPath behaviour — the first element has index 1). So:

- `employee[1]` → **1st** record
- `employee[2]` → **2nd** record
- `employee[3]` → **3rd** record

Your own demo confirms this — you used `employee[3]` and got the **3rd** record, which only works with 1-based indexing. (In Java/Groovy arrays, by contrast, the first item is index 0 — don't mix the two up.)

Small syntax fix: your notes write `employees/employee[2]/name` without the leading slash. As an **absolute** path it should start with `/` : `/employees/employee[2]/name` .

6. Can a Header or Property Hold Multiple Values?

Question: Can we pick multiple records (e.g., Somi and Pavan) into one property or header? How?

Answer: No. A property is a **variable** — it **cannot store multiple values**. The same applies to a message header: **a property or message header stores only a single value at a time.**

If you want multiple records, you need to create **separate fields (one variable per value) in the message header** — e.g., one header for the 2nd employee's name, another for the 3rd.

Demo from Class

1. Postman: Body → raw → XML data (test input)
2. Content Modifier → Message Header → Source Type = **XPath** → value = the employee XPath
3. First test with an incomplete index (empty brackets) → **500 Internal Server Error**
4. Corrected with `employee[3]` → **we get the 3rd record**

Takeaway: A 500 error right after editing an XPath is a strong hint that the XPath itself is malformed (empty brackets, missing

slash, wrong tag name). Check the expression first before suspecting the adapter or the deployment.

7. Task Given

Task → perform an addition operation in Content Modifier. (*Practice assignment from class — try building it in your trial tenant. If you get stuck, send me what you tried and the error, and I'll help you debug.*)

8. Simulation in SAP CPI — Internal Testing

Simulation = internal testing of part of an iFlow, inside the editor, **without deploying** it and without calling real external systems.

Using Functions in the Simulation

Example from class: `concat(//id, //name)` — an XPath function that joins the **id** and **name** values into one string.

Note: XPath function names are **case-sensitive and lowercase** — it is `concat(...)`, not `Concat(...)`.

Simulation Limits (Verified Against SAP Documentation)

Rule from Class	Verified Detail
Simulation → only 10 palette steps (Start → 1 to 10 steps → End)	Correct. Maximum of 10 integration flow steps per simulation run. You must also define a Start Point and an End Point — without an end point, Run Simulation stays greyed out.
"If there is any external call in between the flow, simulation doesn't work"	Needs a small correction — see below.

Correction on external calls: Simulation **does support** External Call steps (Request Reply, Content Enricher, Send) — it just **cannot make the actual call** to the real receiver system. If you **don't provide a mocked response**, the simulation simply passes along the payload from the previous steps (so you won't see a real reply from the external system). So the accurate statement is: *"Simulation won't actually call external systems — to test those parts you either mock the response or test via a real deployed run."*

Other limits worth knowing: maximum payload size is about **1 MB**; on Cloud Foundry tenants the simulation **input** payload must be under **100 KB**; and the **Aggregator** step is not supported in simulation.

9. Standard Expressions (Camel Simple)

CPI provides ready-made standard expressions, all written with `${ ... }`:

Purpose	Expression
Current time	<code>\${date:now:yyyy-MM-dd'T'HH:mm:ss}</code> (<i>the format part is up to you</i>)
Current iFlow name	<code>\${camelId}</code>

Tip: `${camelId}` is very handy for logging/alert emails — it tells you *which* iFlow produced the message without hard-coding the name.

10. Pending Source Types

From the 8 Source Types (Day 10), these are still pending for upcoming classes:

- **Local Variable**
- **Global Variable**
- **Number Range** (*covered below ✓*)

11. Number Range

The Problem It Solves

Suppose an iFlow is triggered **100 times**, but I want to identify the **66th message**. Can we identify it? **Answer: No** — **the sender won't send a serial number**.

Solution: CPI itself adds a **sequence number** to each incoming message using a **Number Range**. In real projects, this is used to **restrict how many times a flow should be hit**, and to **know which time (which run number) the flow was hit last**.

How to Enable a Number Range

Scenario: assigning a number series to the incoming payload message successfully.

1. Go to the **Monitor** tab → **Number Ranges** tile (*this sits under the Manage Stores section we saw on Day 6*)
2. Click **Add**
3. Fill in the fields:

Field	Example / Meaning
Name	NR_test
Description	Short text about what this sequence is for
Minimum Value	Starting number (e.g., 0)
Maximum Value	Upper limit of the sequence
Fixed Length	Pads the number to a set number of digits (e.g., 5 → 00001)
Rotate	If enabled, the sequence restarts from the minimum once it reaches the maximum

Then, in the Content Modifier, choose **Source Type = Number Range** and give the Number Range's name as the value — each message that passes through gets the next number in the series.

Open items in your notes: the "?" marks next to *Maximum Value* and *Fixed Length* weren't filled in — worth asking your trainer what values were used in the demo, or just experiment with small values (e.g., min 1, max 10, rotate on) in your trial tenant to see how it behaves.